Cairo University

Faculty of Engineering

Advanced Tic Tac Toe Game

Software Requirements Specification (SRS)

Supervisor: Dr.Omar Nasr

ID	Name
9220822	مريم سامي راغب عطا
9230707	ماركو شنوده ثابت اسعد
9230373	رنا يوسف عبدالله صالح
9230246	اميرة سامح عبدالحليم سيد
9230384	ريم حسين احمد بسيوني

Table of Contents

1. Introduction

1.1 Purpose

The purpose of this document is to detail all the functional and non-functional requirements for the development of a Tic-Tac-Toe game application. This document will serve as a guide for developers, testers, and stakeholders to understand the system's behavior, performance requirements, and game rules.

1.2 Scope

The Tic-Tac-Toe game allows human players to login , Register, View tutorial website and game rules before playing , play against other human players or AI opponents of varying difficulty levels, and view game histories. The game records and displays the history of moves and game results.

1.3 Definitions, Acronyms, and Abbreviations

- **AI**: Artificial Intelligence
- SRS: Software Requirements Specification
- User: A human player who interacts with the game
- Game Board: A 3x3 grid where players place their symbols ('X' or 'O')

2 Overall Description

2.1 Product Perspective

The Tic-Tac-Toe game is a standalone application designed for entertainment and educational purposes. It can be used by individuals who want to play a quick game of Tic-Tac-Toe against other humans or AI opponents.

2.2 Product Functions

- User login and Register
- View tutorial on wikiHow
- Play Game (Human vs. Human, Human vs. AI(3 levels))
- View Game History
- Save Game History
- Replay My Last Match
- View My History for each account
- Display Game Board and Moves

2.3 User Classes and Characteristics

Regular User: A human player who wants to play Tic-Tac-Toe. Regular users can login and Register, play games, and view their game history.

2.4 OPERATING ENVIRONMENT

- The application will run on any standard computer with a modern C++ compiler.
- The application will use file-based storage for user data and game history.

2.5 DESIGN AND IMPLEMENTATION CONSTRAINTS

- The application will be implemented using C++.
- User data and game history will be stored in plain text files.

2.6 ASSUMPTIONS AND DEPENDENCIES

- Users will have access to a computer with a modern C++ compiler.
- Users will be familiar with basic command-line operations.

3 SPECIFIC REQUIREMENTS

3.1 FUNCTIONAL REQUIREMENTS

3.1.1 User Authentication

Register:

- Users must provide a unique username and a password to sign up.
- The system must check for duplicate usernames and prompt for a different username if necessary.

Login :

- Users must provide their username and password to sign in.
- The system must authenticate the user by checking the provided credentials against stored data.

3.1.2 Gameplay

Start Game:

- Users can start a new game after signing in.
- Users can choose to play against another human or an AI opponent.

- ***Make Move:***
  - Human players must be able to input their move.
  - The system must validate the move (i.e., check if the move is within bounds and the chosen cell is empty).
- ***AI Move:***
  - The system must provide AI players with different difficulty levels (easy , medium, hard).
  - The AI must make a valid move based on its difficulty algorithm.
- ***End Game:***
  - The game ends when a player wins, or the board is full.
  - The system must announce the winner or a draw.

3.1.3 Game History

- ***Save Game History:***
  - The system must save the game history after each game.
  - The history must include player names, the winner .
- ***View account History:***

- Users must be able to view their moves history.
- The number of games played .
- Replay My Last Match
- View My History for each account

3.2 NON-FUNCTIONAL REQUIREMENTS

3.2.1 Performance

1-The game must respond to user inputs within 1 second.

2-The AI must make its move within 2 seconds as maximum.

3-using sleep mode to define delay time 500ms for more realistic game .

```
sleep_for(chrono::milliseconds(500));
```

3.2.2 Usability

1. The game must have a simple and intuitive command-line interface.
2. Instructions and prompts must be clear and concise.

3.2.3 Reliability

- The system must handle incorrect inputs gracefully.
- User data and game history must be stored persistently.

3.2.4 Maintainability

- The code must be well-documented to facilitate future maintenance.
- The design must follow standard coding conventions.

3.3 GAME RULES

- The game is played on a 3x3 grid.
- Players take turns placing their symbol (X or O) in an empty cell.
- The first player to place three of their symbols in a row (horizontally, vertically, or diagonally) wins.
- If all cells are filled and no player has three in a row, the game is a draw.

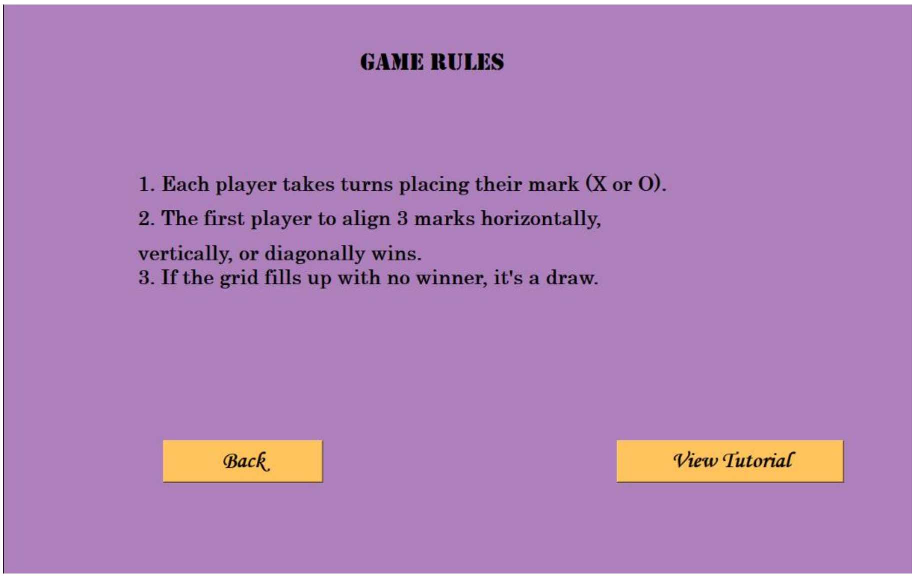
SPECIFIC REQUIREMENTS

4.1 SAMPLE INPUT/OUTPUT

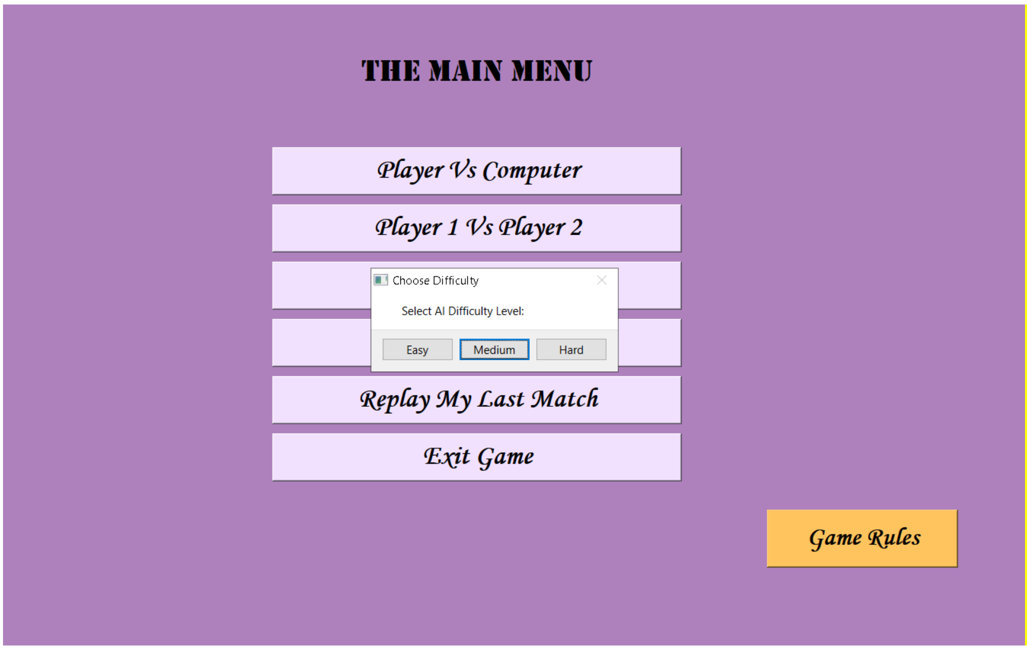
Sample login :

The main menu

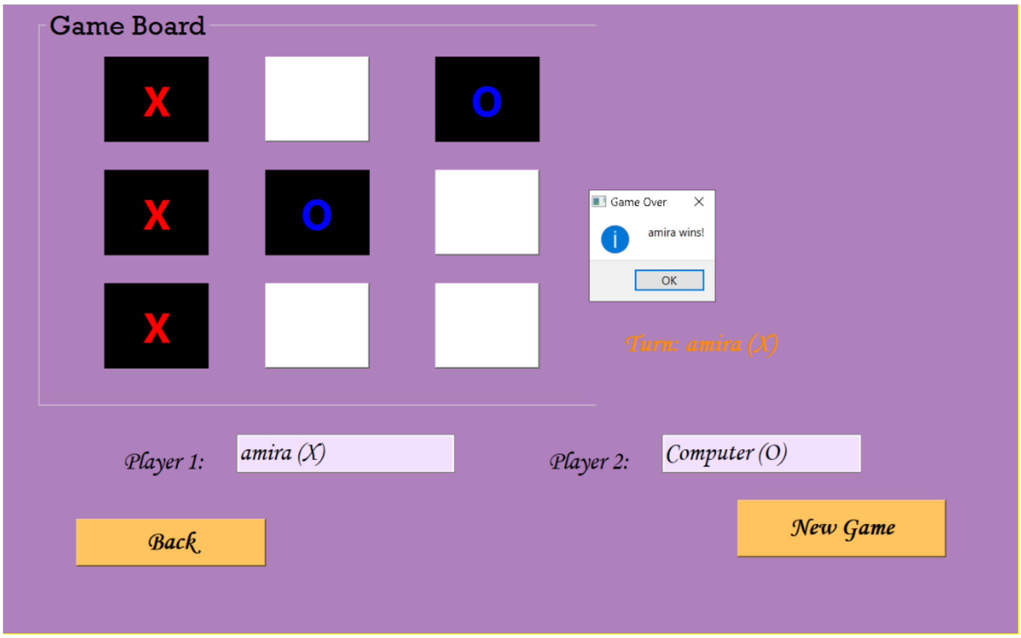
Game rules



AI levels.

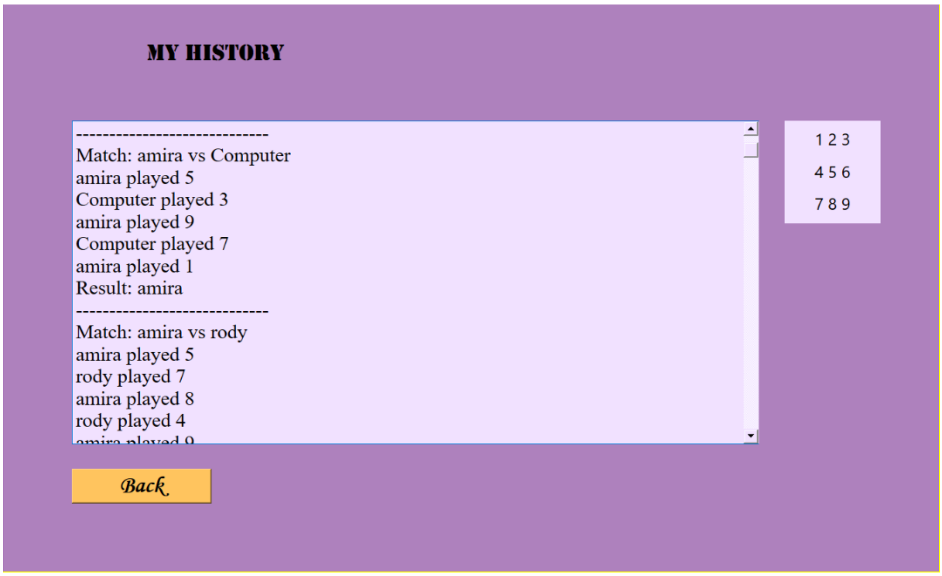


Sample Gameplay:



Leader board

My history



Exit window